COMP 4513
Assignment #2: Single-Page App in React

Version: 1.0 Feb. 24, 2026
Due Monday April 13, 2026 at midnightish

Overview

This assignment provides an opportunity for you to demonstrate your ability to generate a single-page application using React. **You can work alone or in groups of two on this assignment.**

Don't delay finding a partner; if you cannot find a partner to work with, don't ask me to be in a group of 3, you will have to work alone. The assignment is very doable by one person, but it will be a more pleasant experience sharing the work with another. If you are looking for a partner, try posting in the course discord.

Beginning

It is your responsibility to read all the assignment instructions thoroughly and carefully. If you are unclear about something, ask me. But before you do, read the material in question again!

Submit

You must host your site on a working server platform that I can access. Since your React application is a static site, any host that supports static files will work. GitHub Pages, FireBase Hosting, Netlify, Render, and Surge, all provide relatively trouble-free solutions for hosting React apps. If DevOps is your thing, you could use Google Cloud Platform or AWS to create a virtual servers. Or you could use any third-party server that has ftp access.

You must upload the deploy version of Vite using the `vite build` command. You can find instructions for deploying to numerous environments at https://vitejs.dev/guide/build.

I will also need access to the source code, so you will have to make it available to me on GitHub. **If you are hosting on github pages, you will need two Github repos: one with the development version, and one with the production version**! If on a private repo, be sure to invite me.

So, to submit your assignment you must send me an email with three bits of information: 1) the URL of your React app, 2) the development github repo URL, 3) the names of the people in your group.

Grading

This assignment is worth 18% of the course grade. The grade for this assignment will be broken down as follows:

Visual Design, Styling, and Usability	15%
Programming Design + Documentation	10%
Functionality (follows requirements)	70%
Extras	+10%

Requirements

This assignment consists of a single HTML page (hence single-page application or SPA) with the following functionality:

1. You must make use of the `Vite` starting files and structure. The filename for your starting file should be `index.html` (`Vite` does this automatically).
2. You are encouraged to use a third-party React UI Component library and/or CSS Framework such as Tailwind (it would be good to have that on your resume). Every UI component library has a learning curve; generally, the more customizable it is, the bigger the learning curve. From what I can see, the most popular in 2025 (in no order) are: MUI, Mantine UI, ant, daisy UI, React Bootstrap, shadcn, and heroUI. The last two take a different approach in that you only install the components you need and allow for more visual customization via tailwind.
3. This assignment consists of a variety of views (remember this is a single-page application, so it is best to think of the assignment consisting of different views). In the remainder of this assignment specification, I have provided a sketch of the basic layout of each view.

   These sketches do not show the precise styling (or even the required layout); rather they show functionality and content. I will be expecting your pages to look significantly more polished and attractive than these sketches! A lot of the 15% design mark will be based on my assessment of how much effort you made on the styling and design. Ideally, the React UI Component library does a lot of the design work for you!
4. If you've used JS that you've found online, I expect you to indicate that in your documentation AND in your about dialog. Provide a URL of where you found it in the documentation. Failure to do so might result in a zero mark, so give credit for other people's work!

5. **You have three choices when it comes to how you are going to retrieve the data.**
    a. Use your assignment 1 web API in Node running on a hosted node server that returns this same data. You can then host your React application on any static service (such as github pages). You may also decide to host your React application on the same server as your Node web api. This will require making some changes to your assign 1 script so it can handle static requests (see Lab13a).
    b. If you have used supabase then you can use its client features (for instance see lab14b). This means you don't need a hosted server for the API. You only need to host your React application on any static service (such as github pages)

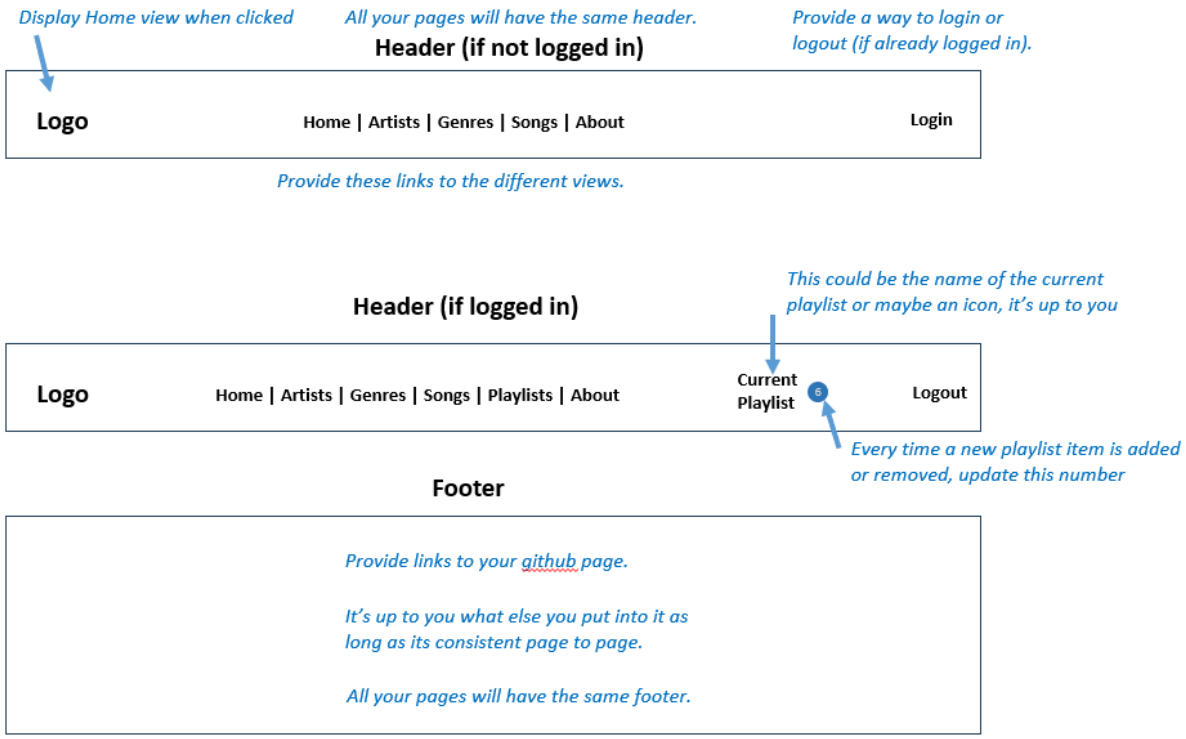
    There is one complication: this assignment will require you to implement some type of login system. You can make it purely a potemkin village, a façade that looks like a functioning login system but it doesn't really do any authentication. But to earn the extra credit marks, you must implement a functioning authentication system which will likely require you to have a node server or to use supabase.

    The API will take some time to retrieve this data the first time. Display a loading animation (there are many free animated GIF available) until the data is retrieved. Simply display the image just before the fetch and then hide it after fetch is successful.

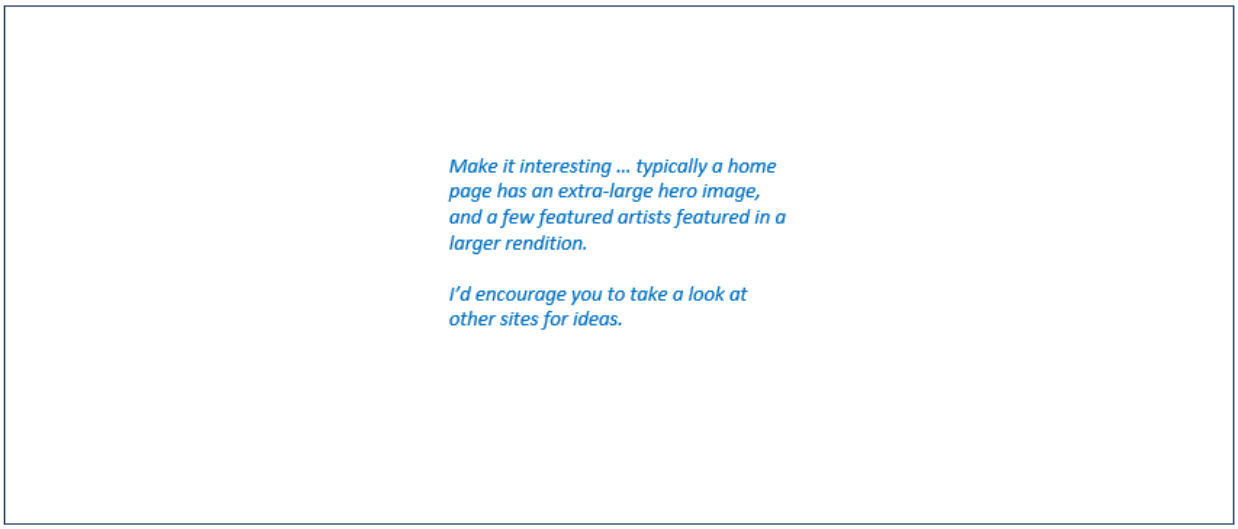
6. The assignment should have five main views (**Home, Artists / Genres Entry, Single Artist, Single Genre, Songs, Single Songe, Login,** and **Playlists**) and a dialog/popup/drawer for the **About** information. When the program first starts, display the **Home** view.

7. You already have the data source from Assignment One.

8. Header / Footer. All your views will contain the same header / footer. Another way of saying this is that you will be changing the content *between* the header and footer. The image below demonstrates the functionality required. The links are not URLs to other html files; instead, they indicate that your SPA must display a different view.



9. **Home View**. You will implement a page with similar functionality as that described below. As you can see, I've left it up to you in terms of what content to put on this page.



10. **Artists / Genres Entry View**. Both artists and genres should have a landing page/view of some sort, with functionality similar to that shown below. Each artist or genre name should take the user to **Browse View** with the relevant songs for that artist or genre. Notice that I haven't provided you with any images, but the data for th4 artist includes a URL that you can use as the image source.

There are lots of ways of visually displaying this information. I'd encourage you to take a look at online stores for ideas.

For the artists, you can display the artist image. For the genres, you'll likely need some type of placeholder service or use some type of AI generator.

11. **Browse View.** This is the heart of the assignment. Your view must be able to allow user to browse songs that match a variety of different criteria and which has the functionality listed below.

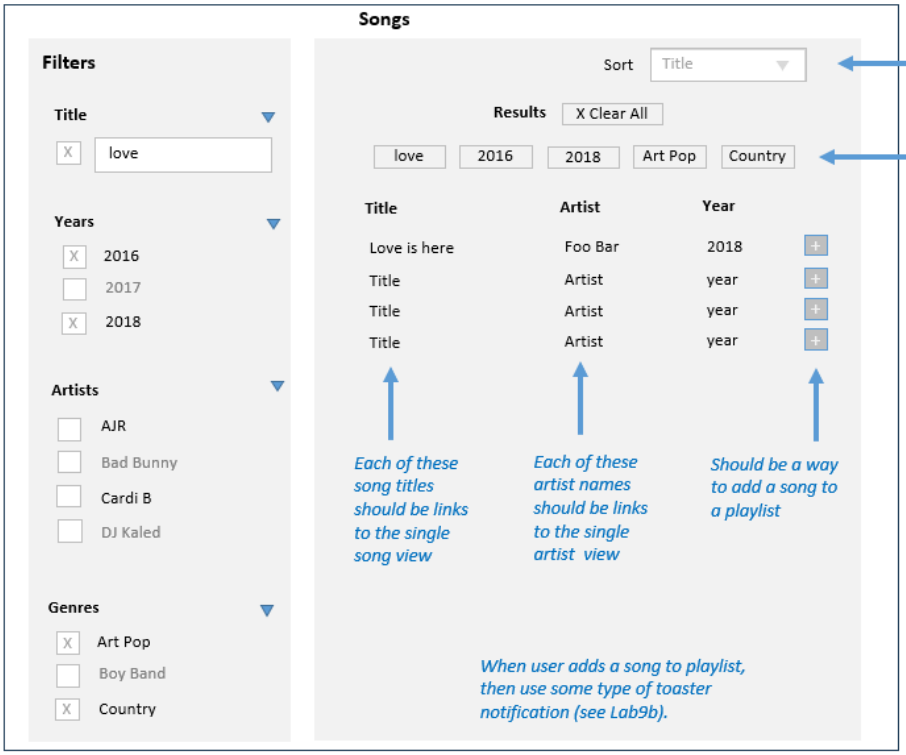
The filters should be additive (ANDs). Be sure to provide visual indication of which ones have been selected. I've indicated two possibilities here (bold and checkboxes).

In this example, you would display all the songs with the word "love" in the title with a year or either 2016 or 2018 who genre is either Art Pop or Country.

Need to provide a way to remove a filter.

Every time a new filter is added or removed, update the list of matching songs. If there are no matches, tell the user that this is the case.

I'd encourage you to take a look at online stores or music sites for ideas.

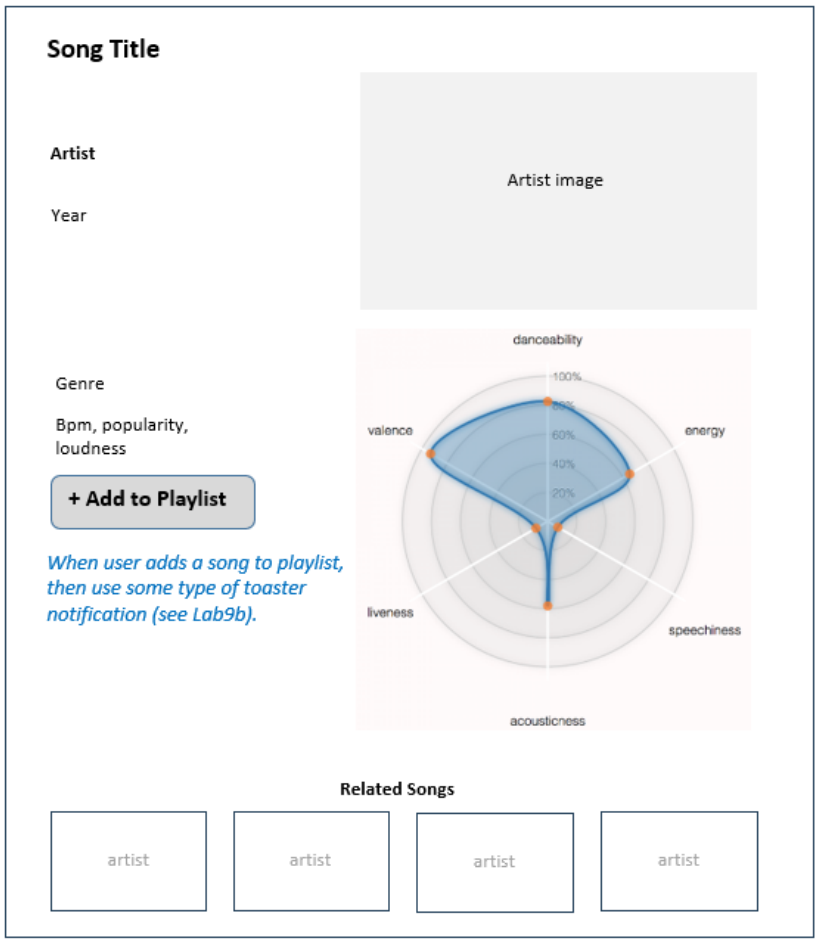


Should also be a way to change the sort order between: song title, year, and artist name. The default should be song title.

Somewhere you need to show the current filters (and provide easy way to clear them). Here you can see the five filters in place. Perhaps you allow the user to remove these filters selectively here.

There are lots of ways of visually displaying this information. I'd encourage you to take a look at other sites for ideas.

12. Single Song View. Display details for a single song with the following functionality. You can add a song to the current playlist; you could also instead allow the user to add it to a specific playlist.

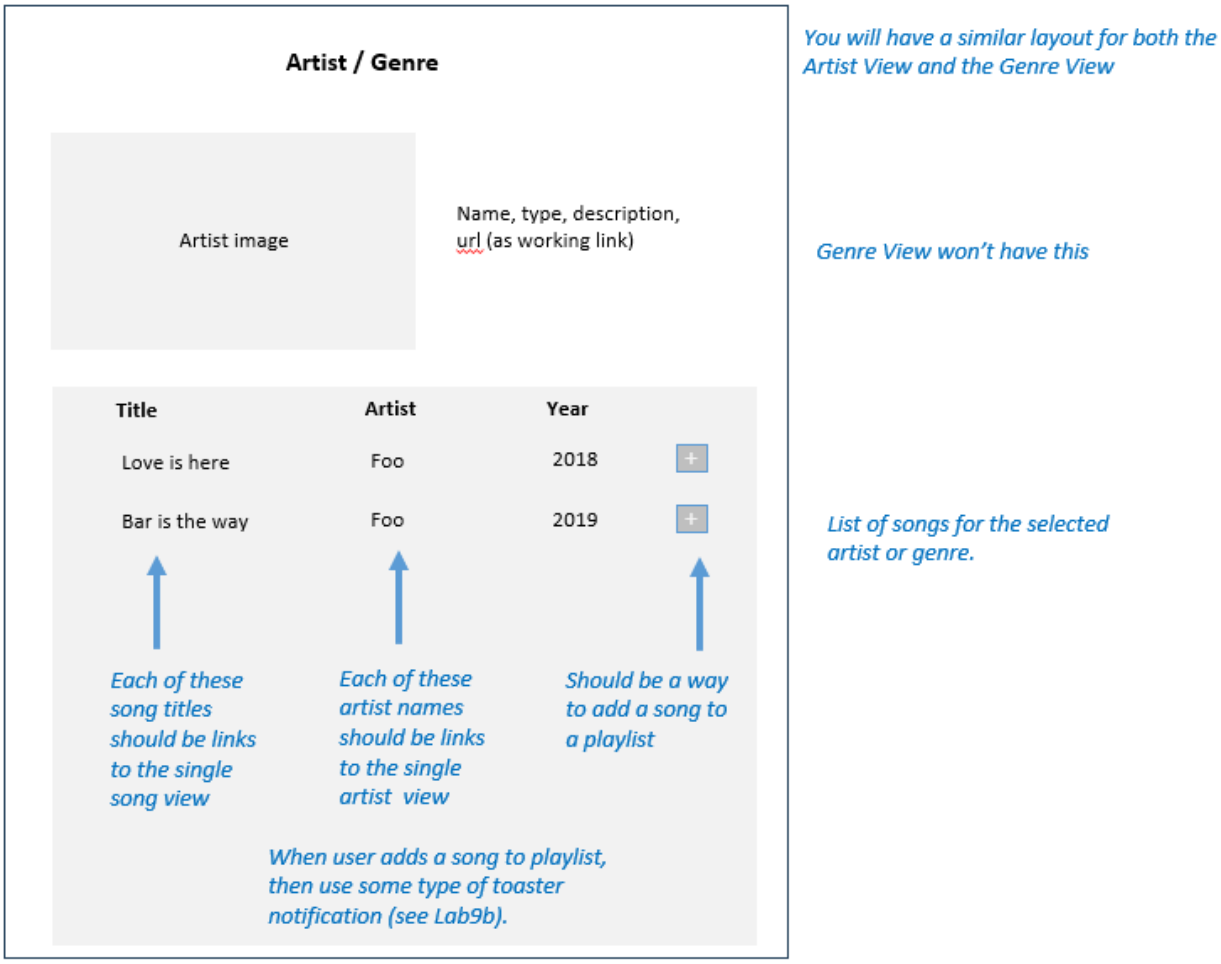


Display the song information in any way you'd like but take advantage of contrast, alignment, etc

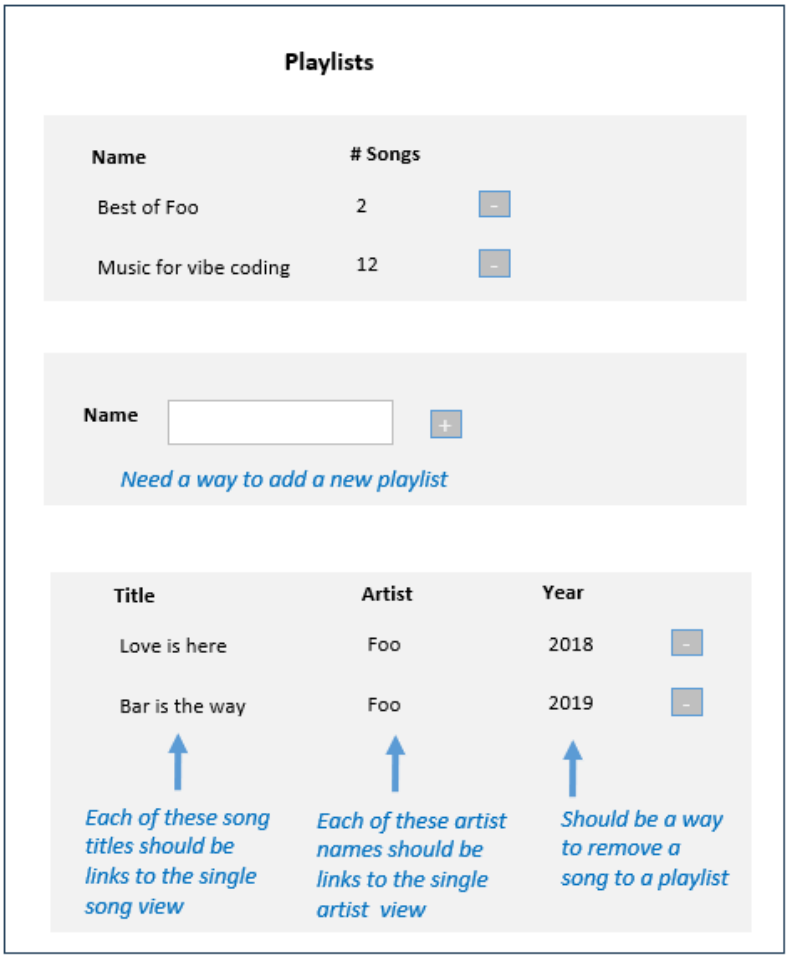
Display these analytic values in a radar chart

What makes a related song?
For this assignment, display some number of songs whose analytic values have a similar sum to the top 3 analytic categories as this song. In this example (look at the chart), you would look for songs with similar valence, danceability, and energy values.

13. Single Artist / Single Genre View. Display details for a single artist or genre along with matching songs.



14. Playlist View. View existing playlists, modify playlists, and display content of different playlists. The playlists must persist; that is, if I close the application and then restart it, the playlists will still be there. You will implement this feature by maintaining the playlists on the server (that is, you will need a CRUD API for playlists).

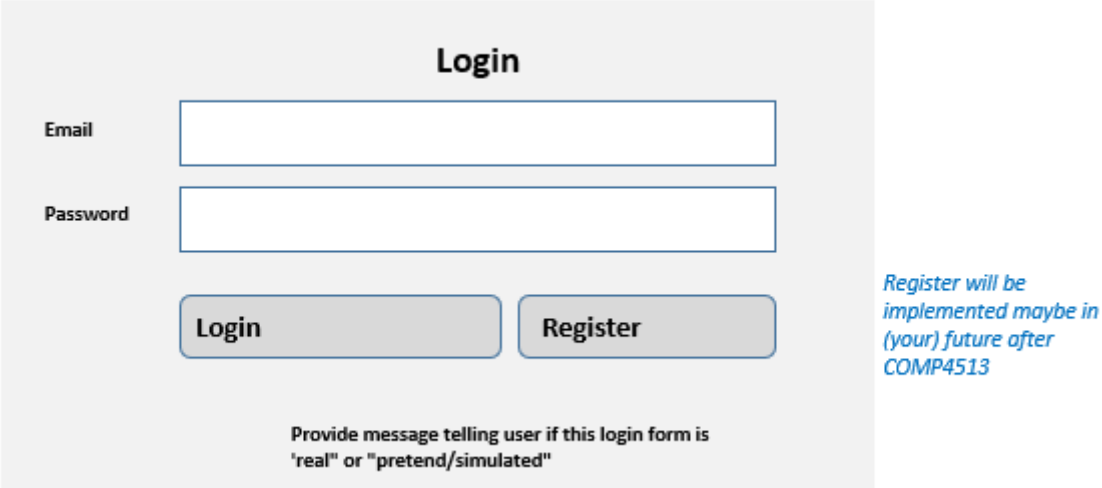


Display a list of existing playlists for this user. Each playlist should have a name. Also display the number of songs in the playlist, plus provide a way to delete a playlist.

When the user clicks a playlist, it becomes the current playlist. Its contents will be shown below; as well, the Add-to-Playlist buttons will modify this playlist.

When user removes a song from a playlist, then use some type of toaster notification (see Lab9b).

15. **Login View.** When the user clicks the Login link in the header, then display the login view/dialog.



At the very least, you will have to implement a “pretend” authentication system. That is, populate the form with valid user credentials and then “record” internally that the user is now logged in. The Log Out option in the menu should change this status to not logged in.

Depending on whether there is time available, I will ask you to implement a real login authentication system using Node or even using supabase auth. In the last few years, there wasn’t enough time.

It is very likely, however, there won’t be enough time this year either, and so you will not need to implement the actual authentication, but will just simulate it by jumping to the Home View when the login button is clicked.

If you decide to expand on this project during the spring or summer, implementing this functionality will make your assignment a fantastic portfolio piece!

The 10% Extra Credits are available if your authentication system actually works using Node or Supabase.

16. **About Us Dialog/Popup.** Display an about dialog that has info about the assignment, the technologies (libraries) you are using, info about yourself, a link to the github repo, etc. Most React UI component libraries provide this functionality.

17. There is duplicate functionality in the song lists on the different views. This **must** be implemented with a single component.

Testing

Every year students lose many easy marks because they didn't read the requirements carefully enough. When your assignment is getting close to done, I would recommend going through each requirement step and carefully evaluate whether your assignment satisfies each specified requirement. After you upload to your host, test again!!!

Submitting and Hosting

Your assignment source code must reside on GitHub and reside on a working public server (in this case GitHub Pages).

GitHub Pages works in conjunction with git and github. You push your html/css/images to github repo; and then push to the host. The instructions for doings so can be found at:

```
https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site
```

It is up to you whether you want your pages to be public (available to the world) or private (available to only those with access to your github repo).

If you are using a private repo, you must add me as a collaborator!

I would strongly recommend getting your hosting to work a few days before the due date. It's okay if your assignment is still not complete at that point: the idea here is to make sure hosting works ahead of time!

When your hosting is working and the assignment is ready to be marked, then send me an email with the following information:

- The URL of the home page of the site on github pages.
- The URL of the github repo so that I can mark the source code. If your repo is private, then add me as a collaborator.

Hints

1. Remember that JSON and JavaScript are case sensitive. For instance, it is easy to type in `Price` and it won't work because the JSON field is actually `price`.
2. Most years, students tend to get low marks on the design side of the assignment. I would recommend looking at other sites on the internet and examine how they present data. Notice the use of contrast (weight, color, etc) and spacing.
3. Using React means there is not a lot of programming design to assess. I will expect proper document for each component. If I see overly large components, I will likely reduce the design mark since a large component is harder to maintain.
4. You will need to find some way of handling routes. React Router should do the trick.
5. If working in a group, there are enough use cases in this assignment to keep everyone busy. You may want to set up a division of labour early on.